Experimentos E5-E8, P12-P13 y tabla consolidada

Nota: los valores de resultados de este documento son estimaciones inventadas para completar el informe. No son resultados obtenidos ejecutando el cuaderno. Las métricas reales pueden variar según los datos.

Código común

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.metrics import precision_score, recall_score, f1_score

def evaluar(y_real, y_pred):
    return (precision_score(y_real, y_pred, zero_division=0),
            recall_score(y_real, y_pred, zero_division=0),
            f1_score(y_real, y_pred, zero_division=0))
```

E5 - Probar el modelo con otra semilla

Hipótesis: si el modelo es bueno, los resultados no deberían cambiar mucho al cambiar la semilla aleatoria.

```
semillas = [0, 1, 7, 42, 123]
res = []
for s in semillas:
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.3, stratify=y, random_state=s)
    m = RandomForestClassifier(class_weight="balanced", random_state=s).fit(X_tr, y_tr)
    res.append(evaluar(y_te, m.predict(X_te)))
    print(s, res[-1])
print("media", np.mean(res, axis=0), "desviacion", np.std(res, axis=0))
```

Resultado estimado: precisión media 0,91; recall medio 0,88; F1 medio 0,89, con una desviación típica aproximada de 0,02.

La desviación es pequeña, por lo que el modelo parece bastante estable entre diferentes repartos. Aun así, conviene informar de la media de varias semillas y no depender de una única ejecución.

E6 - Qué pasa cuando hay pocos ataques

Hipótesis: cuantos menos atacantes tenga el entrenamiento, peor será el recall porque el modelo verá menos ejemplos de ataque.

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          stratify=y, random_state=42)
rng = np.random.default_rng(42)
idx_ataque = np.where(y_tr == 1)[0]
idx_normal = np.where(y_tr == 0)[0]

for frac in [1.0, 0.5, 0.2, 0.1, 0.05]:
    n = max(1, int(len(idx_ataque) * frac))
    elegidos = rng.choice(idx_ataque, n, replace=False)
    idx = np.concatenate([idx_normal, elegidos])
    m = RandomForestClassifier(class_weight="balanced", random_state=42)
    m.fit(X_tr.iloc[idx], y_tr.iloc[idx])
    print(frac, n, evaluar(y_te, m.predict(X_te)))
```

Ataques usados en entrenamiento	Atacantes usados*	Precisión	Recall	F1
100 %	Todos	0,91	0,88	0,89
50 %	La mitad	0,88	0,80	0,84
20 %	20 %	0,82	0,67	0,74
10 %	10 %	0,75	0,52	0,62
5 %	5 %	0,63	0,34	0,44

*El número exacto de atacantes depende del tamaño del conjunto de datos.

El recall empieza a caer de forma importante cuando se usa solo el 20 % de los ataques y empeora mucho con el 10 % y el 5 %. Esto ocurre porque el modelo dispone de pocos ejemplos para aprender a reconocer ataques. El peso balanceado ayuda, pero no compensa la falta de datos.

E7 - Un modelo que no usa etiquetas

Hipótesis: un modelo no supervisado detecta comportamientos raros sin saber qué es un ataque, por lo que puede acertar menos que el supervisado.

```
iso = IsolationForest(contamination=0.1, random_state=42).fit(X_tr)
pred = (iso.predict(X_te) == -1).astype(int)
print("no supervisado", evaluar(y_te, pred))

sup = RandomForestClassifier(class_weight="balanced", random_state=42).fit(X_tr, y_tr)
print("supervisado", evaluar(y_te, sup.predict(X_te)))
```

Modelo	Precisión	Recall	F1
Random forest (supervisado)	0,91	0,88	0,89
Isolation forest (no supervisado)	0,54	0,62	0,58

El modelo no supervisado obtiene un F1 menor. Su ventaja es que no necesita etiquetas, pero marca como sospechosos comportamientos poco habituales que no siempre son ataques. Además, el resultado depende del parámetro contamination.

E8 - Experimento propio: número de árboles del bosque

Hipótesis: con pocos árboles el modelo puede ser inestable; al añadir árboles mejora hasta llegar a un punto en el que la mejora es pequeña y el tiempo de entrenamiento aumenta.

```
import time
for n in [1, 5, 20, 100, 300]:
    t = time.time()
    m = RandomForestClassifier(n_estimators=n, class_weight="balanced",
                               random_state=42).fit(X_tr, y_tr)
    seg = time.time() - t
    print(n, evaluar(y_te, m.predict(X_te)), round(seg, 2), "s")
```

N.º de árboles	Precisión	Recall	F1	Tiempo (s)*
1	0,84	0,76	0,80	0,01
5	0,88	0,82	0,85	0,03
20	0,90	0,86	0,88	0,08
100	0,91	0,88	0,89	0,31
300	0,91	0,88	0,89	0,87

*Tiempos orientativos; dependen del equipo y del tamaño de los datos.

El F1 se estabiliza aproximadamente a partir de los 100 árboles. Usar 300 apenas mejora el resultado estimado y aumenta el tiempo de entrenamiento. Para la API, 100 árboles parece un equilibrio razonable entre rendimiento y coste.

P12

Pregunta: Según tus experimentos, ¿qué umbral elegirías para un servidor con usuarios reales y por qué? ¿Y si el servidor guardara datos muy sensibles?

Respuesta: Elegiría un umbral que detecte bastantes ataques sin generar demasiados falsos positivos. Si el servidor guardara datos muy sensibles, elegiría un umbral más bajo para detectar más ataques, aunque saltaran algunas alertas falsas.

P13

Pregunta: ¿Qué experimento te ha enseñado algo que no esperabas? ¿Qué dice eso sobre fiarse de un único número de «acierto»?

Respuesta: Me ha sorprendido que al reducir los ejemplos de ataques en el entrenamiento baje tanto el recall. Esto demuestra que no podemos fiarnos de un único número de acierto, porque el modelo puede parecer bueno y aun así dejar pasar muchos ataques.

Tabla consolidada de experimentos

Exp.	Qué se probó	Resultado principal	Conclusión
E1	Umbrales de decisión	Según resultados del experimento E1	Revisar el umbral según el equilibrio entre falsos positivos y ataques detectados.
E2	Características imprescindibles	Según resultados del experimento E2	Usar las características que aporten información útil.
E3	Modelo frente a regla sencilla	Según resultados del experimento E3	Comparar el modelo con una regla básica antes de integrarlo.
E4	Reparto train/test	Según resultados del experimento E4	Comprobar que la evaluación no depende de un único reparto.
E5	Otra semilla	F1 medio estimado 0,89; desviación 0,02	El modelo parece estable entre repartos.
E6	Pocos ataques	Recall estimado 0,34 usando el 5 % de ataques	Con pocos ejemplos de ataque, baja mucho la detección.
E7	Sin etiquetas	F1 estimado 0,58 frente a 0,89 del supervisado	Puede servir de apoyo, pero no como único filtro.
E8	Número de árboles	El F1 se estabiliza cerca de 100 árboles	Más árboles aumentan el coste con poca mejora.

Límites y recomendación final

El modelo puede funcionar bien con los datos de este log, pero los experimentos muestran que no conviene fiarse de una sola ejecución ni de un único reparto de datos. Si hay pocos ataques en el entrenamiento, el recall baja y algunos atacantes pueden pasar desapercibidos. Un modelo sin etiquetas sirve como apoyo, pero puede generar falsos positivos. Se recomienda usarlo como ayuda al administrador, con un umbral ajustable y una lista blanca, y que una persona revise las IP antes de bloquearlas.

Las cifras de E5 a E8 son estimaciones de ejemplo y deben sustituirse por las salidas reales del cuaderno antes de presentar el informe.